Artefato 7 – Projeto Físico de Banco de Dados

Sistema de apoio à gestão de requisitos com histórias de usuário | 08/11/2026

Este documento apresenta o projeto físico do banco de dados relacional da aplicação de apoio à gestão de requisitos. O banco foi modelado para armazenar usuários, projetos, participação de membros, backlogs, histórias de usuário, épicos, critérios de aceitação e informações de priorização.

A aplicação utiliza o SQLite como mecanismo de persistência e o Django ORM como camada de acesso aos dados. O esquema é criado e evoluído por meio de migrações versionadas. O diagrama físico deve ser inserido na seção 1 deste documento, no espaço reservado.

1 Diagrama físico do banco de dados

O diagrama deve apresentar as tabelas da aplicação, suas colunas principais, as chaves primárias e estrangeiras e as cardinalidades dos relacionamentos. As tabelas de autenticação do Django aparecem somente quando participam de relacionamentos do domínio, como o usuário que é dono ou membro de um projeto.

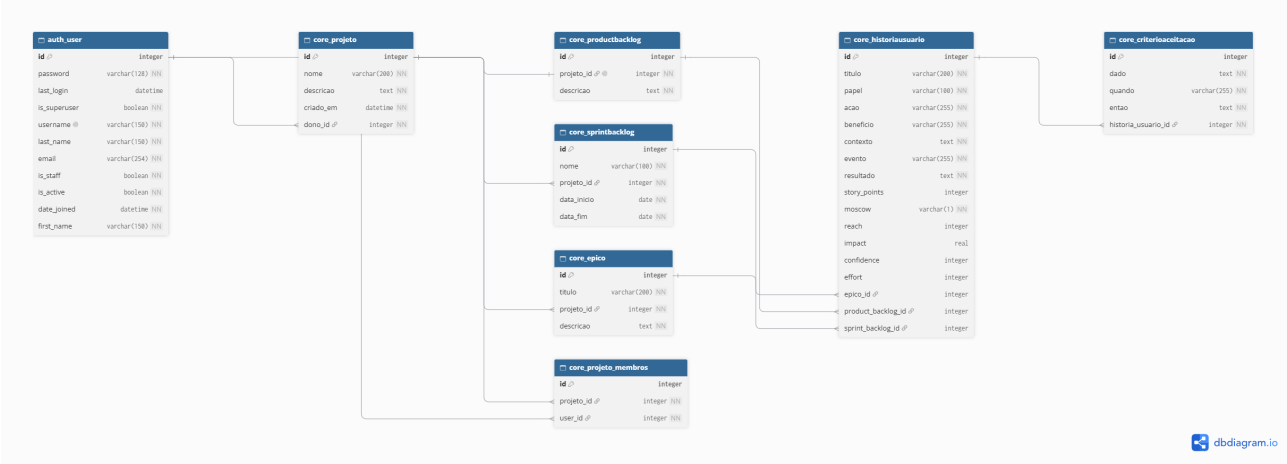


Figura 1: Diagrama físico do banco de dados.

2 Convenções do modelo físico

As tabelas utilizam uma coluna `id` como chave primária, salvo quando uma restrição adicional é indicada. As colunas terminadas em `_id` representam chaves estrangeiras. As cardinalidades são descritas da seguinte forma:

- **1:1**: um registro de uma tabela se relaciona com no máximo um registro da outra;
- **1:N**: um registro da primeira tabela pode se relacionar com vários registros da segunda;
- **N:N**: vários registros dos dois lados se relacionam por meio de uma tabela associativa.

Uma história de usuário possui duas colunas opcionais para representar sua localização: `product_backlog_id` e `sprint_backlog_id`. Essas colunas são mutuamente exclusivas: uma história deve estar em um único backlog por vez.

3 Entidades e tabelas

3.1 Tabela `auth_user`

Propósito. Armazena as contas de usuários autenticados. É uma tabela fornecida pelo mecanismo de autenticação do Django e é utilizada pelo domínio para identificar donos e membros de projetos. A senha é armazenada pelo Django como hash, e não como texto puro.

Coluna	Tipo	Nulo	Chave	Descrição
`id`	INTEGER	Não	PK	Identificador do usuário.
`username`	VARCHAR(150)	Não	Única	Nome de identificação utilizado pelo Django.
`password`	VARCHAR(128)	Não	–	Hash da senha do usuário.
`first_name`	VARCHAR(150)	Não	–	Nome do usuário.
`last_name`	VARCHAR(150)	Não	–	Sobrenome do usuário.
`email`	VARCHAR(254)	Não	–	Endereço de e-mail da conta.
`is_active`	BOOLEAN	Não	–	Indica se a conta está ativa.
`is_staff`	BOOLEAN	Não	–	Indica acesso administrativo ao Django.
`is_superuser`	BOOLEAN	Não	–	Indica privilégios de superusuário.
`date_joined`	DATETIME	Não	–	Data de criação da conta.
`last_login`	DATETIME	Sim	–	Data do último login.

Relacionamentos: um usuário pode ser dono de vários projetos e pode participar de vários projetos como membro por meio de `core_projeto_membros`.

3.2 Tabela `core_projeto`

Propósito. Representa um projeto de software que reúne backlogs, histórias, épicos e participantes.

Coluna	Tipo	Nulo	Chave	Descrição
`id`	INTEGER	Não	PK	Identificador do projeto.
`nome`	VARCHAR(200)	Não	–	Nome do projeto.
`descricao`	TEXT	Não	–	Descrição e objetivo do projeto.
`criado_em`	DATETIME	Não	–	Data e hora de criação.
`dono_id`	INTEGER	Não	FK	Referência a `auth_user.id`.

Relacionamentos:

- cada projeto possui um único dono;
- um usuário pode ser dono de vários projetos;
- cada projeto possui um Product Backlog;
- cada projeto pode possuir várias Sprints e vários épicos;
- membros são relacionados pela tabela associativa.

3.3 Tabela `core_projeto_membros`

Propósito. Implementa a participação de usuários em projetos. É a tabela associativa do relacionamento muitos-para-muitos entre usuários e projetos.

Coluna	Tipo	Nulo	Chave	Descrição

id	INTEGER	Não	PK	Identificador do vínculo.
projeto_id	INTEGER	Não	FK	Referência a core_projeto.id.
user_id	INTEGER	Não	FK	Referência a auth_user.id.

Existe uma restrição única sobre o par (projeto_id, user_id). Assim, o mesmo usuário não pode ser adicionado duas vezes ao mesmo projeto.

Relacionamento: auth_user N:N core_projeto, resolvido por esta tabela.

3.4 Tabela core_productbacklog

Propósito. Armazena o Product Backlog de um projeto, incluindo sua descrição ou objetivo.

Coluna	Tipo	Nulo	Chave	Descrição
id	INTEGER	Não	PK	Identificador do Product Backlog.
projeto_id	INTEGER	Não	FK, UNIQUE	Projeto ao qual o backlog pertence.
descricao	TEXT	Não	–	Descrição ou objetivo do produto.

A restrição de unicidade em projeto_id garante o relacionamento **1:1** entre projeto e Product Backlog. Um Product Backlog pode possuir várias histórias de usuário.

3.5 Tabela core_sprintbacklog

Propósito. Representa uma Sprint Backlog pertencente a um projeto. Um projeto pode possuir várias Sprints, cada uma com seu próprio período.

Coluna	Tipo	Nulo	Chave	Descrição
id	INTEGER	Não	PK	Identificador da Sprint.
nome	VARCHAR(100)	Não	–	Nome da Sprint.
projeto_id	INTEGER	Não	FK	Projeto ao qual a Sprint pertence.
data_inicio	DATE	Não	–	Data de início da Sprint.
data_fim	DATE	Não	–	Data de término da Sprint.

Existe uma restrição que exige data_fim >= data_inicio. Uma Sprint pode possuir várias histórias. Ao excluir uma Sprint, suas histórias retornam ao Product Backlog do mesmo projeto.

3.6 Tabela core_epico

Propósito. Representa uma funcionalidade ampla que agrupa histórias de usuário relacionadas dentro de um projeto.

Coluna	Tipo	Nulo	Chave	Descrição
id	INTEGER	Não	PK	Identificador do épico.
titulo	VARCHAR(200)	Não	–	Nome do épico.
projeto_id	INTEGER	Não	FK	Projeto ao qual o épico pertence.
descricao	TEXT	Não	–	Descrição da funcionalidade abrangida.

Um projeto possui vários épicos. Um épico pode possuir várias histórias, mas uma história pode estar vinculada a no máximo um épico por vez. Ao excluir um épico, as histórias permanecem e ficam sem vínculo com ele.

3.7 Tabela `core_historiausuario`

Propósito. Armazena as histórias de usuário, seus vínculos com backlogs e épicos, suas estimativas e seus dados de priorização.

Coluna	Tipo	Nulo	Chave	Descrição
`id`	INTEGER	Não	PK	Identificador da história.
`titulo`	VARCHAR(200)	Não	–	Título curto da história.
`papel`	VARCHAR(100)	Não	–	Papel do usuário no formato “Como um”.
`acao`	VARCHAR(255)	Não	–	Ação desejada no formato “Eu quero”.
`beneficio`	VARCHAR(255)	Não	–	Benefício esperado no formato “Para”.
`contexto`	TEXT	Não	Legado	Campo legado “Dado”.
`evento`	VARCHAR(255)	Não	Legado	Campo legado “Quando”.
`resultado`	TEXT	Não	Legado	Campo legado “Então”.
`story_points`	INTEGER	Sim	–	Estimativa de tamanho da história.
`moscow`	VARCHAR(1)	Não	–	Prioridade Must, Should, Could ou Won’t.
`reach`	INTEGER	Sim	–	Alcance estimado no RICE.
`impact`	REAL	Sim	–	Impacto estimado no RICE.
`confidence`	INTEGER	Sim	–	Confiança: 50, 80 ou 100.
`effort`	INTEGER	Sim	–	Esforço utilizado no cálculo RICE.
`epico_id`	INTEGER	Sim	FK	Épico associado, quando existir.
`product_backlog_id`	INTEGER	Sim	FK	Product Backlog associado.
`sprint_backlog_id`	INTEGER	Sim	FK	Sprint Backlog associado.

As chaves estrangeiras de backlog são opcionais individualmente, mas mutuamente exclusivas. Portanto, a história deve estar no Product Backlog ou em uma Sprint Backlog, nunca nos dois simultaneamente.

Os campos `story_points` e `effort` são independentes. O primeiro representa a estimativa para planejamento de Sprint; o segundo é o divisor utilizado no cálculo RICE:

$$RICE = \frac{Reach \times Impact \times Confidence}{Effort}$$

3.8 Tabela `core_criterioaceitacao`

Propósito. Armazena critérios de aceitação individuais no formato Dado/Quando/Então. A separação em uma tabela própria permite que uma história possua vários critérios.

Coluna	Tipo	Nulo	Chave	Descrição
`id`	INTEGER	Não	PK	Identificador do critério.
`dado`	TEXT	Não	–	Contexto inicial do critério.
`quando`	VARCHAR(255)	Não	–	Evento ou ação que dispara o critério.
`entao`	TEXT	Não	–	Resultado esperado.
`historia_usuario_id`	INTEGER	Não	FK	História à qual o critério pertence.

O relacionamento é de um-para-muitos: uma história pode possuir vários critérios, mas cada critério pertence a uma única história. A exclusão da história exclui seus critérios associados.

4 Relacionamentos entre as tabelas

Tabela de origem	Cardinalidad	Tabela relacionada	Regra
auth_user	1:N	core_projeto	Um usuário pode ser dono de vários projetos.
auth_user	N:N	core_projeto	Relação intermediada por core_projeto_membros.
core_projeto	1:1	core_productbacklog	Cada projeto possui no máximo um Product Backlog.
core_projeto	1:N	core_sprintbacklog	Um projeto pode possuir várias Sprints.
core_projeto	1:N	core_epico	Um projeto pode possuir vários épicos.
core_productbacklog	1:N	core_historiausuario	O Product Backlog agrupa histórias ainda não alocadas em Sprint.
core_sprintbacklog	1:N	core_historiausuario	Uma Sprint reúne histórias selecionadas para uma iteração.
core_epico	1:N	core_historiausuario	Um épico agrupa histórias relacionadas.
core_historiausuario	1:N	core_criterioaceitacao	Uma história pode possuir vários critérios.

5 Regras de integridade e restrições

O banco e a aplicação utilizam as seguintes regras para preservar a consistência dos dados:

- um projeto possui exatamente um dono;
- um usuário não pode ser repetido como membro do mesmo projeto;
- um projeto possui no máximo um Product Backlog;
- um projeto pode possuir vários Sprint Backlogs;
- a data de término da Sprint não pode ser anterior à data de início;
- uma história não pode estar simultaneamente no Product Backlog e em uma Sprint;
- uma história não pode ser criada sem backlog;
- o épico e o backlog associados a uma história devem pertencer ao mesmo projeto;
- uma história pode possuir no máximo um épico;
- excluir um épico não exclui suas histórias;
- excluir uma Sprint devolve suas histórias ao Product Backlog;
- excluir uma história exclui seus critérios de aceitação;
- Story Points e Effort aceitam somente a escala definida para o projeto;
- Confidence aceita somente os valores 50, 80 e 100;
- Reach não pode possuir valor negativo.

6 Tabelas técnicas do framework

Além das tabelas descritas neste documento, o Django cria tabelas técnicas para controlar migrações, sessões, permissões, grupos e registros do painel administrativo. Entre elas estão `django_migrations`, `django_session`, `django_content_type`, `django_admin_log`, `auth_group` e `auth_permission`.

Essas tabelas não representam entidades específicas do domínio da aplicação. Por isso, podem ser omitidas do diagrama principal ou apresentadas em uma área separada como tabelas de suporte do framework.

7 Conclusão

O modelo físico utiliza um banco relacional porque os dados possuem relações estruturadas e regras de integridade claras. A tabela de projeto centraliza os backlogs e épicos; as histórias representam os requisitos; e os critérios de aceitação foram separados em uma tabela própria para permitir múltiplos critérios por história.

As chaves estrangeiras mantêm os vínculos entre as entidades, enquanto as restrições e validações impedem combinações inválidas, como uma história pertencer simultaneamente a dois backlogs ou um épico ser associado a uma história de outro projeto.